

Arquitectura de computadoras

Compuertas lógicas



CUCEI

Clave: IL365 NRC: 200204 Seccion: D03 Calendario: 2026B

Ariel Rojas Garcia

221930873

1. INTRODUCCIÓN

1.1 La ALU dentro del procesador

Uno de los componentes esenciales de todo procesador es la **Unidad Aritmético-Lógica (ALU)**, encargada de ejecutar las operaciones aritméticas (suma, resta, incremento, comparación) y lógicas (AND, OR, NOT, XOR, entre otras) que requiere el procesamiento de las instrucciones de un programa.

Internamente, la ALU no es un bloque monolítico: está formada por subcircuitos especializados, cada uno dedicado a un tipo de operación, cuyos resultados se seleccionan mediante un multiplexor controlado por una señal de operación proveniente de la unidad de control. Esta práctica se centra específicamente en el **bloque de operaciones lógicas**, implementado a nivel de un solo bit.

1.2 Lógica digital y álgebra de Boole

La **lógica digital** es la rama de la electrónica que estudia y diseña circuitos que operan con señales discretas, representadas mediante dos estados: 0 (nivel bajo) y 1 (nivel alto). Su fundamento matemático es el **álgebra de Boole**, formulada por George Boole en el siglo XIX y aplicada al diseño de circuitos de conmutación por Claude Shannon en 1937.

El álgebra de Boole define tres operaciones fundamentales sobre variables binarias:

- Conjunción (**AND**), denotada como AB
- Disyunción (**OR**), denotada como $A + B$
- Negación (**NOT**), denotada como A'

Las **compuertas lógicas** son la realización física de estas operaciones mediante circuitos electrónicos, típicamente contruidos con transistores en tecnología CMOS. Combinando compuertas se construyen circuitos combinacionales (sumadores, multiplexores, decodificadores, comparadores) y, agregando elementos de memoria, circuitos secuenciales (flip-flops, registros, contadores). En conjunto, estos bloques conforman todos los componentes internos de un procesador.

1.3 Verilog y el diseño mediante HDL

Verilog es un lenguaje de descripción de hardware (HDL, *Hardware Description Language*) desarrollado en la década de 1980 y posteriormente estandarizado por el Instituto de Ingenieros Eléctricos y Electrónicos (IEEE) bajo la norma IEEE 1364. Se utiliza para modelar, simular,

verificar y sintetizar circuitos digitales, desde compuertas individuales hasta procesadores completos.

Una diferencia conceptual fundamental respecto a un lenguaje de programación de software es que **en un HDL no se escribe una secuencia de instrucciones que se ejecutan una tras otra, sino que se describe hardware que existe de manera simultánea y concurrente**. Varias asignaciones continuas dentro de un módulo no ocurren "una después de otra": representan circuitos que operan al mismo tiempo.

La unidad básica de organización en Verilog es el módulo, delimitado por las palabras clave **module** y **endmodule**, que declara sus puertos de entrada y salida y describe internamente el comportamiento o la estructura del circuito. Los módulos pueden instanciarse dentro de otros módulos, lo que permite construir diseños jerárquicos.

Verilog permite describir un mismo circuito en distintos **niveles de abstracción**:

- **Estructural (nivel de compuerta)**: se instancian directamente primitivas o submódulos y se conectan entre sí. Es el más cercano al circuito físico.
- **Flujo de datos (dataflow)**: se utilizan asignaciones continuas “assign” con expresiones booleanas. Es el más habitual para lógica combinacional.
- **Conductual (behavioral)**: se describe el comportamiento deseado mediante bloques procedurales “always”, sin especificar la estructura interna.

El proceso de trabajo consiste en escribir la descripción del circuito, verificarla mediante un **banco de pruebas** (*testbench*) que aplica estímulos y observa las salidas, y finalmente —cuando se busca una implementación física— sintetizarla para obtener una red de compuertas real.

1.4 Tipos de sintaxis posibles para los operadores lógicos en Verilog

a) Nivel de compuerta (estructural, con primitivas)

Verilog incluye primitivas predefinidas para las compuertas básicas. La sintaxis es **primitiva nombre_instancia (salida, entrada1, entrada2);** — el primer puerto siempre es la salida.

b) Flujo de datos (dataflow, con assign y operadores bit a bit)

Se emplean asignaciones continuas y operadores booleanos. Es la forma más compacta y la más común para lógica combinacional.

c) Conductual (*behavioral*, con bloques **always**)

Se describe el comportamiento dentro de un bloque procedural sensible a cualquier cambio en las entradas. En este estilo, las salidas deben declararse como tipo **reg**.

También podría describirse mediante estructuras **if** o **case** que reproduzcan directamente la tabla de verdad, aunque para este caso resulta innecesariamente extenso.

2. OBJETIVO

Objetivo general

Comprender e implementar los fundamentos de la lógica combinacional aplicados al diseño del bloque lógico de una ALU, mediante la descripción en lenguaje Verilog de un único módulo que integre las siete compuertas lógicas principales, verificando su correcto funcionamiento por simulación.

Objetivos particulares

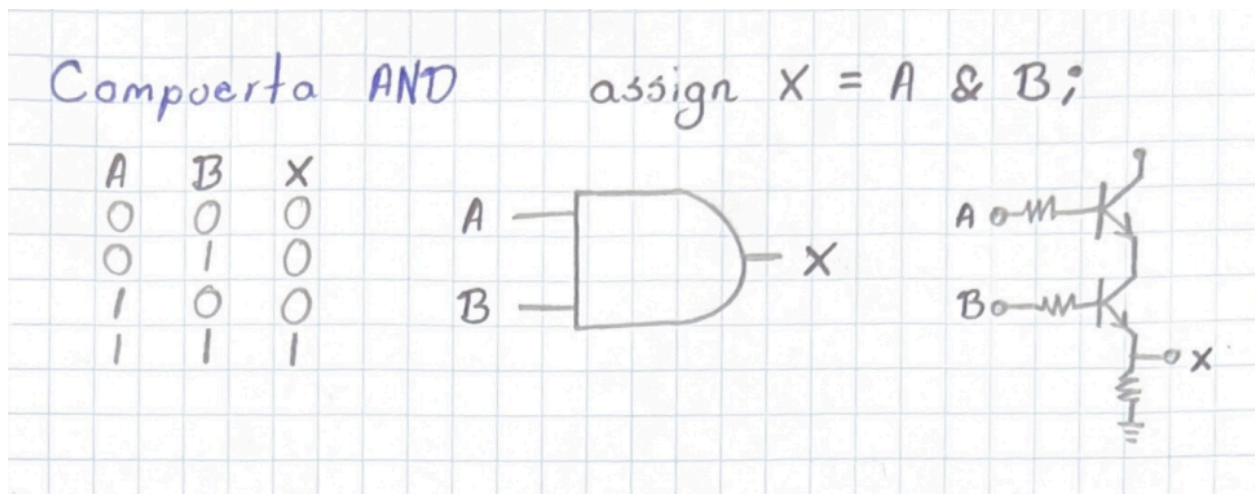
1. Verificar la correcta descarga, instalación y funcionamiento del compilador y simulador de HDL que se utilizará durante el semestre (ModelSim), resolviendo cualquier error de configuración del software.
2. Investigar la sintaxis de los operadores de lógica combinacional en Verilog, identificando y comparando los distintos estilos de descripción disponibles: estructural, de flujo de datos y conductual.
3. Documentar, para cada una de las siete compuertas lógicas principales, su símbolo normalizado, su tabla de verdad completa, su expresión booleana, su circuito y su sintaxis correspondiente en Verilog.
4. Diseñar e implementar el módulo **Compuertas** en un solo archivo con extensión **.v**, con dos entradas de un bit (**A**, **B**) y siete salidas de un bit (**C_AND**, **C_OR**, **C_NOT**, **C_NAND**, **C_NOR**, **C_XOR**, **C_XNOR**).

3. DESARROLLO

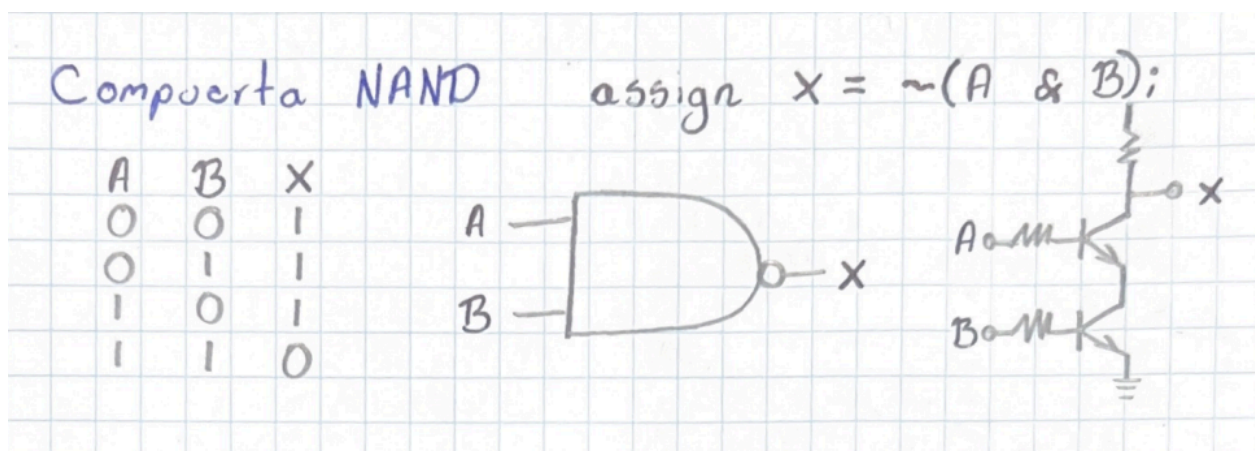
3.1 Las 7 compuertas lógicas

Se elaboró de forma manual una ficha por cada una de las siete compuertas, con el fin de afianzar la comprensión de su comportamiento antes de describirlas en Verilog.

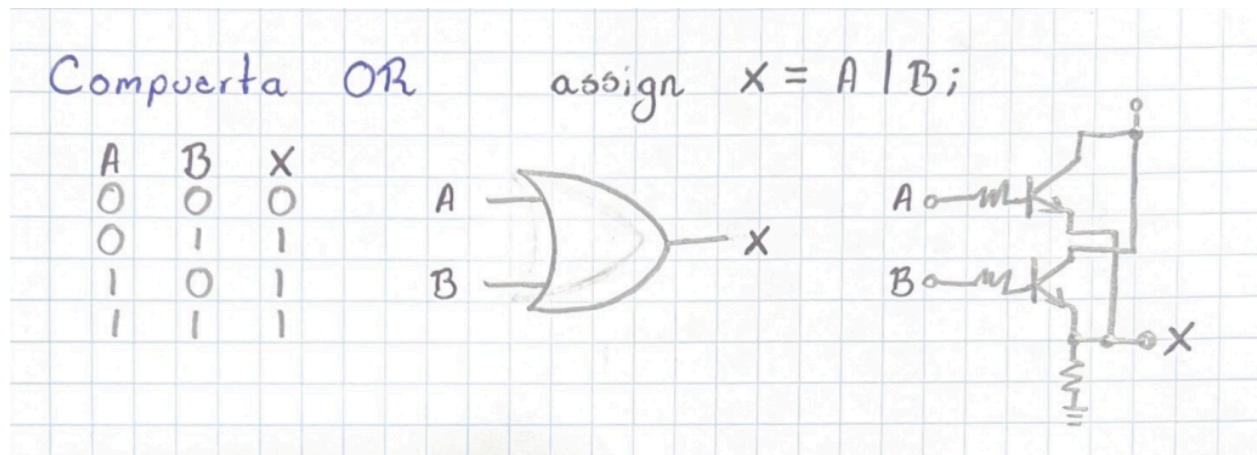
AND



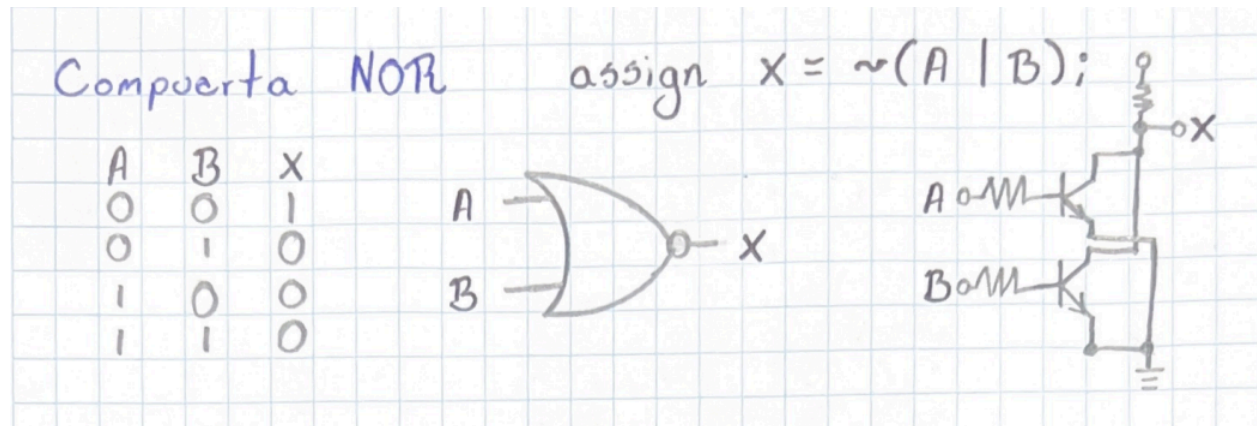
NAND



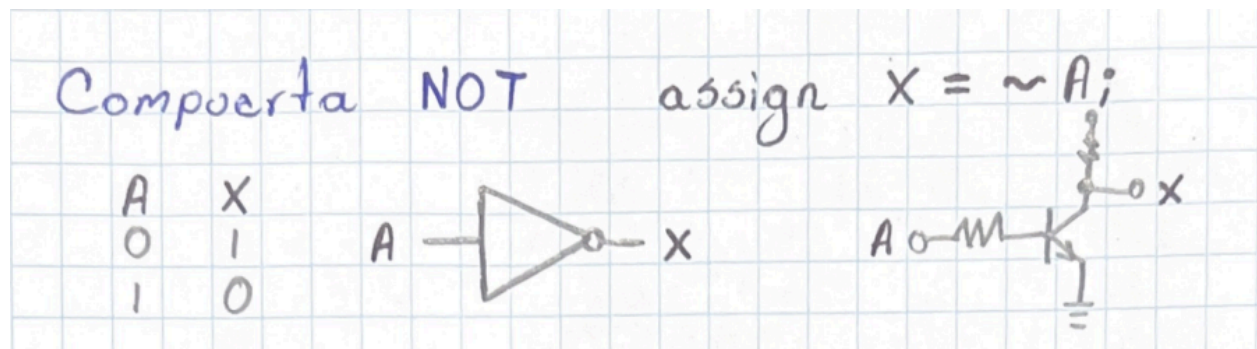
OR



NOR



NOT



XOR

Compuerta XOR			assign $X = A \wedge B;$
A	B	X	
0	0	0	
0	1	1	
1	0	1	
1	1	0	

XNOR

Compuerta XNOR			assign $X = \sim(A \wedge B);$
A	B	X	
0	0	1	
0	1	0	
1	0	0	
1	1	1	

3.2 Instalación y verificación del entorno de trabajo

- Descarga e instalación de **ModelSim**.
- Procedimiento seguido en ModelSim: creación del proyecto, agregado del archivo fuente: **Compuertas.v**, compilación, inicio de la simulación y adición de señales a la ventana de ondas.

3.3 Diseño e implementación del módulo **Compuertas**

- **Definición del módulo:** nombre **Compuertas**, dos puertos de entrada de un bit (A, B) declarados como **input**, y siete puertos de salida de un bit (C_AND, C_OR, C_NOT, C_NAND, C_NOR, C_XOR, C_XNOR) declarados como **output**.

- **Justificación del estilo de descripción elegido:** Se uso el flujo de datos `assign`, por ser el más directo y legible para lógica puramente combinacional, y porque garantiza que el circuito resultante no incluya elementos de memoria.
- **La compuerta NOT:** al tener una sola entrada, se aplica únicamente sobre `A`.
- **Inclusión del código fuente completo**

None

```
`timescale 1ns/1ns

module Compuertas (
    input A,
    input B,
    output C_AND,
    output C_OR,
    output C_NOT,
    output C_NAND,
    output C_NOR,
    output C_XOR,
    output C_XNOR
);

assign C_AND  = A & B;
assign C_OR   = A | B;
assign C_NOT  = ~A;
assign C_NAND = ~(A & B);
assign C_NOR  = ~(A | B);
assign C_XOR  = A ^ B;
assign C_XNOR = ~(A ^ B);

endmodule
```

3.4 Resultados y validación

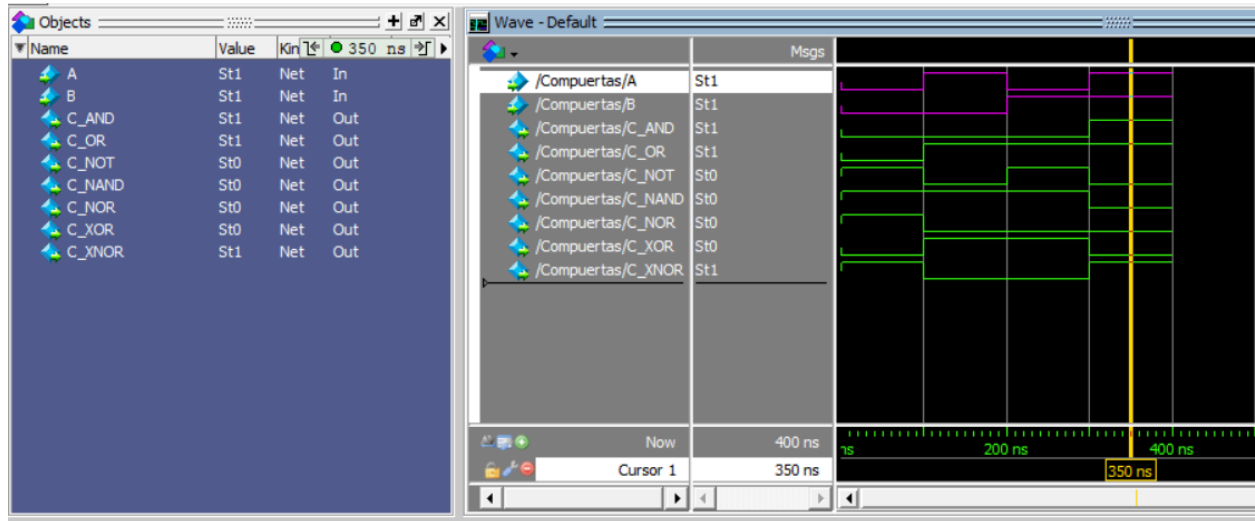


Figura 8. Ventana Wave de resultados obtenidos en la simulación

4. CONCLUSIÓN

Esta práctica me permitió tener mi primer contacto real con un lenguaje de descripción de hardware, y creo que lo más valioso no fue la complejidad del circuito —que en realidad es sencillo— sino el cambio de mentalidad que implica. Al principio escribía el código como si fuera un programa normal, pensando que las líneas se ejecutaban una tras otra, y me costó trabajo asimilar que en Verilog no estoy escribiendo instrucciones sino describiendo circuitos que existen todos al mismo tiempo. Entender que las siete asignaciones del módulo representan siete compuertas físicas operando simultáneamente, y no siete pasos secuenciales, fue el aprendizaje más importante que me dejó la actividad.

5. REFERENCIAS

- Mano, M. M., & Ciletti, M. D. *Diseño Digital*. Pearson Educación.
- Tocci, R. J., Widmer, N. S., & Moss, G. L. *Sistemas Digitales: Principios y Aplicaciones*. Pearson Educación.
- Palnitkar, S. *Verilog HDL: A Guide to Digital Design and Synthesis*. Prentice Hall.
- Ciletti, M. D. *Advanced Digital Design with the Verilog HDL*. Pearson.
- Floyd, T. L. *Fundamentos de Sistemas Digitales*. Pearson Educación.
- IEEE. *IEEE Standard for Verilog Hardware Description Language (IEEE Std 1364)*. IEEE Computer Society.
- Shannon, C. E. (1938). *A Symbolic Analysis of Relay and Switching Circuits*. *Transactions of the American Institute of Electrical Engineers*, 57(12), 713–723.